Task 1: Guided Activity, Tip calculator with MVVM:

```
import Foundation
struct TipCalculation {
    var bill: Double
    var tipPercent: Double
    var people: Int

    var tipAmount: Double {
        bill * tipPercent / 100
    }

    var total: Double {
        bill + tipAmount
    }

    var perPerson: Double {
        total / Double(max(people, 1))
    }
}

class TipViewModel {
    var bill: Double = 1200
    var tipPercent: Double = 10
    var people: Int = 4

    var calculation: TipCalculation {
        TipCalculation(
            bill: bill,
            tipPercent: tipPercent,
            people: people
```

```
        )
    }

    var tipAmountText: String {
        String(format: "%.2f", calculation.tipAmount)
    }

    var totalText: String {
        String(format: "%.2f", calculation.total)
    }

    var perPersonText: String {
        String(format: "%.2f", calculation.perPerson)
    }
}

func showResults() {
    let vm = TipViewModel()

    print("Tip Calculator")
    print("Bill Amount: ₹\(String(format: "%.2f", vm.bill))")
    print("Tip Amount: ₹\(vm.tipAmountText)")
    print("Total Bill: ₹\(vm.totalText)")
    print("People: \(vm.people)")
    print("Per Person: ₹\(vm.perPersonText)")
}

showResults()
```

output:

Output

```
Tip Calculator
Bill Amount: ₹1200.00
Tip Amount: ₹120.00
Total Bill: ₹1320.00
People: 4
Per Person: ₹330.00
```

Task 2: Practice Task, Classify the code:

```
import Foundation

struct Student {

let name: String

let marks: Int

}

class StudentViewModel {

var students: [Student] = []


func loadStudents() {

  students = [

    Student(name: "Arun", marks: 85),

    Student(name: "Priya", marks: 35),

    Student(name: "Kumar", marks: 65)

  ]

}


func sortStudentsByMarks() {

  students.sort { $0.marks > $1.marks }

}


func gradeText(for student: Student) -> String {

  return student.marks >= 40 ? "Pass" : "Fail"

}
```

```
}
func displayStudents(viewModel: StudentViewModel) {
for student in viewModel.students {
let color = student.marks < 40 ? "Red" : "Black"

   print("Name: \(student.name)")
   print("Marks: \(student.marks)")
   print("Grade: \(viewModel.gradeText(for: student))")
   print("Color: \(color)")
   print("----------------")
}

}
let vm = StudentViewModel()
vm.loadStudents()
vm.sortStudentsByMarks()
displayStudents(viewModel: vm)
```

output:

```
Output

Name: Arun
Marks: 85
Grade: Pass
Color: Black
----------------
Name: Kumar
Marks: 65
Grade: Pass
Color: Black
----------------
Name: Priya
Marks: 35
Grade: Fail
Color: Red
----------------
```

Task 3: Testability Task, Move validation out of the screen:

```
import Foundation
```

```
class LoginViewModel {
var email = ""
var password = ""
var isValid: Bool {
   return email.contains("@") &&
        email.contains(".") &&
        password.count >= 8
}
}
let vm = LoginViewModel()
vm.email = "[user@gmail.com](mailto:user@gmail.com)"
vm.password = "password123"
print("Check 1:", vm.isValid)

vm.email = "usergmail.com"
vm.password = "password123"
print("Check 2:", vm.isValid)

vm.email = "user@gmailcom"
vm.password = "password123"
print("Check 3:", vm.isValid)

vm.email = "[user@gmail.com](mailto:user@gmail.com)"
vm.password = "pass12"
print("Check 4:", vm.isValid)

vm.email = "usergmailcom"
vm.password = "abc"
print("Check 5:", vm.isValid)
```

output:

Output

```
Check 1: true
Check 2: false
Check 3: false
Check 4: false
Check 5: false
```